7.2.3. Detecting nodes that can’t be reclaimed using hazard pointers

The term *hazard pointers* is a reference to a technique discovered by Maged Michael.[1] They are so called because deleting a node that might still be referenced by other threads is hazardous. If other threads do indeed hold references to that node and proceed to access the node through that reference, you have undefined behavior. The basic idea is that if a thread is going to access an object that another thread might want to delete, it first sets a hazard pointer to reference the object, thus informing the other thread that deleting the object would indeed be hazardous. Once the object is no longer needed, the hazard pointer is cleared. If you’ve ever watched the Oxford/ Cambridge boat race, you’ve seen a similar mechanism used when starting the race: the cox of either boat can raise their hand to indicate that they aren’t ready. While either cox has their hand raised, the umpire may not start the race. If both coxes have their hands down, the race may start, but a cox may raise their hand again if the race hasn’t started and they feel the situation has changed.

[1] “Safe Memory Reclamation for Dynamic Lock-Free Objects Using Atomic Reads and Writes,” Maged M. Michael, in *PODC ’02: Proceedings of the Twenty-first Annual Symposium on Principles of Distributed Computing* (2002), ISBN 1-58113-485-1.

When a thread wishes to delete an object, it must first check the hazard pointers belonging to the other threads in the system. If none of the hazard pointers reference the object, it can safely be deleted. Otherwise, it must be left until later. Periodically, the list of objects that have been left until later is checked to see if any of them can now be deleted.

Described at such a high level, it sounds relatively straightforward, so how do you do this in C++?

Well, first off you need a location in which to store the pointer to the object you’re accessing, the *hazard pointer* itself. This location must be visible to all threads, and you need one of these for each thread that might access the data structure. Allocating them correctly and efficiently can be a challenge, so you’ll leave that for later and assume you have a function